

Reporte de validación semántica:

En esta etapa del proyecto, hemos pasado de simplemente revisar si el código está bien escrito (sintaxis) a revisar si lo que se escribió tiene coherencia lógica (semántica). No basta con que una instrucción como `c = a + b`; sea válida por su estructura, si no que el compilador ahora debe verificar que las variables `a` y `b` existan y que sus tipos de dato sean compatibles entre sí para evitar errores en tiempo de ejecución.

Validación

Para lograr esto, implementamos una **tabla de símbolos**. Esta tabla funciona como la "memoria" del compilador durante el análisis:

Registro: Cada vez que el programa encuentra una declaración (como `int x`);, se guarda el nombre y el tipo en la tabla.

Ámbitos (Scopes): El analizador distingue entre variables globales y las que viven dentro de una función (locales).

Búsqueda: Cuando el código intenta usar una variable, el analizador busca en la tabla. Si no la encuentra, lanza un error de "Variable no definida".

Pruebas y resultados

Ejemplo 1: Detección de incompatibilidad y funciones faltantes En este caso, probamos un código donde se intenta usar una función `suma()` que no ha sido declarada. Además, realizamos una operación aritmética entre un `float` y un `int`.

Resultado: El analizador detectó correctamente que la función no existe y emitió un aviso (warning) sobre la posible pérdida de precisión al asignar un número con decimales a una variable entera.

Código Ejemplo 1:

```
int main(){
float a;
int b;
int c;
c = a+b;
c = suma(8,9);
}
```

Resultado:

```
ST=2   tok=(23,$      ) act=R1      stack=[0 2] in=[$]
ST=1   tok=(23,$      ) act=aceptacion R1 stack=[0 1] in=[$]
Archivo valido sintácticamente.

--- Iniciando Análisis Semántico ---
Se encontraron 2 problemas semánticos:
-> Aviso Semántico: Asignando float a int en 'c'.
-> Error Semántico: La función 'suma' no ha sido definida.
```

Ejemplo 2: Validación de funciones definidas En esta prueba, incluimos la definición de la función antes de llamarla. El analizador recorre el árbol sintáctico, registra la función en la tabla de símbolos y permite su uso más adelante en el main.

Resultado: Se confirma la validez sintáctica y se procede con la validación de tipos en las asignaciones.

Código Ejemplo 2:

```
int a;
int suma(int a, int b){
return a+b;
}

int main(){
float a;
int b;
int c;
c = a+b;
c = suma(8.5,9.9);
}
```

Resultado:

```
ST=1   tok=(23,$      ) act=aceptacion R1 stack=[0 1] in=[$]
Archivo valido sintácticamente.

--- Iniciando Análisis Semántico ---
Se encontraron 2 problemas semánticos:
-> Aviso Semántico: Asignando float a int en 'c'.
-> Aviso Semántico: Asignando float a int en 'c'.
```